

Features:

- Code review automation
- Easy integration with development workflow
- Tackles technical debt by knowing insights
- Prevents security related issues
- Sets up code quality standards

Limitations:

- No good authentication and authorisation in the server
- Does not have integration with continuous integration automation tools
- Not widely used as it is a small community

Pricing: There is a free open source plan available. There is a PRO/self-hosted version available at a price point of US\$ 15 per month.

Veracode

Veracode analyses only security issues and is developed on the SaaS model. It uses binary code/bytecode, ensuring 100 per cent test coverage. It is considered one of the best tools if you want to write secure code and avoid any security loopholes or flaws in it.

This tool performs code analysis via mediums like IDE scans, pipeline scans and policy scans. It creates and reports code for quality assessment inspections.

Languages supported: Java (Java SE, Java EE, JSP), .NET (C#, ASP.NET, VB.NET), JavaScript (including AngularJS, Node.js, and jQuery), Scala, Python, PHP, Ruby on Rails, ColdFusion, Swift, C/C++, COBOL, Visual Basic 6, RPG, and many more

Features:

- Single platform to manage application security
- IDE scan, quick results in the pipeline
- Governance dashboard available for reporting
- Covers security standards

Limitations:

- Creation of customised rule set is not possible
- UI is not so user-friendly

Pricing: Pricing of the licence is based on the size of the project. To get a pricing quote you need to submit the form on the Veracode official website.

DeepScan

DeepScan is a leading-edge static analysis tool which performs semantic code analysis beyond what Lint does. It is used to check feasible runtime errors and quality issues rather than coding conventions. It helps to detect issues like the use of inconsistent null checks, use of implicit type conversion, assignment with the same values, and unreachable code. Integration of DeepScan with your GitHub repositories is possible in order to get a quality insight into your GitHub project. It is very efficient because its analysis speed is very fast (about 7000 lines/sec).

Languages supported: JavaScript, TypeScript, React, and Vue.js

Features:

- Bug tracking
- Dashboards available for audit report generation
- Quality check grades ('Poor', 'Normal', and 'Good')
- Precise detection of issues (lower false alarm rate)
- Collaboration
- Support for continuous integration

Limitations: Limited language support

Pricing: Free for open source projects. Paid plans start from US\$ 7-8 per seat/month.

DeepSource

DeepSource helps to identify and fix bug risks, anti-patterns, performance related issues, and security related flaws on every commit and pull request. It has good integration with GitHub, GitLab and Bitbucket ALM tools.

Languages supported: Supports Python, Go, Ruby, JavaScript, Terraform, Dockerfile, SQL and more

Features:

- Continuous analysis
- Team collaboration
- Single file configuration: Enables DeepSource by adding `.deepsource.toml` configuration file to your repository
- Tracks code metrics
- Blocks pull requests by category of issues

Limitations: No support for PHP

Pricing: Free version available with limited functionality. Paid versions with more functionalities are available for large teams.

In the second article under this two-part series, we will discuss using SonarQube for static code analysis because it is available for multiple languages, is open source, has IDE integration and is available for use with continuous integration (CI) pipeline in Jenkins, Azure DevOps and other CI tools. We will also explore SonarQube integration with the Jenkins pipeline. 

References

- [1] SonarQube Documentation, <https://docs.sonarqube.org/latest/>
- [2] Hands-on Pipeline as Code with Jenkins, <https://www.amazon.com/Hands-Pipeline-Code-Jenkins-Implementation/dp/9389898609>

By: Ankita Patil

The author has written the book 'Hands-on Pipeline as Code with Jenkins: CI/CD Implementation for Mobile, Web, and Hybrid Applications Using Declarative Pipeline in Jenkins'.